Motor de historias interactivas: *La última transmisión*

Parcial práctico de Ingeniería de Software

Alan Chiquillo Torres (T00089445)

Universidad Tecnológica de Bolívar

Docente: Aisner Jose Marrugo Juliao

15 de octubre de 2026

Repositorio: https://github.com/8alanct-hub/sistema_de_historias_interectativas

Versión anterior al cambio: tag `v1.0` (commit 50bc184)

Versión posterior al cambio: tag `v2.0` (commit c89ccda)

Entrega individual aprobada por el docente.

Motor de historias interactivas: *La última transmisión*

Fundamentos

Un colectivo de escritores prepara ficción interactiva para un festival universitario. No saben programar, pero necesitan crear y modificar historias en archivos sin cambiar el programa, que las opciones dependan de lo ocurrido antes y detectar los errores del contenido antes de que alguien juegue. La solución es un motor de consola que carga historias en JSON, valida su contenido y permite jugarlas (Universidad Tecnológica de Bolívar, 2026a).

Los actores son el lector, que juega; la escritora principal, que edita los JSON; el diseñador de la experiencia, que define condiciones y consecuencias, y la coordinadora del festival, que necesita saber qué corregir y que el lector nunca vea un cierre inesperado. El alcance incluye cargar y validar historias, jugar en consola mostrando solo las opciones habilitadas, recordar el estado, rechazar entradas inválidas, reconocer finales y bloqueos, reiniciar y deshacer la última decisión. No incluye traducción, interfaz visual, retroceso de varios niveles ni los límites de la guía (cuentas, base de datos, guardado entre ejecuciones).

Se identificaron dos riesgos: mensajes de error poco claros para la escritora, mitigado con un mensaje en español para cada tipo de error que indica la causa y el ID afectado, y un cierre inesperado ante un caso no contemplado, mitigado validando toda entrada en la partida y capturando cualquier error en la interfaz. Los atributos de calidad se priorizaron pensando en el lector: confiabilidad, facilidad de uso y claridad de errores, en ese orden. Como decisión de responsabilidad profesional sobre datos, el motor no pide ni guarda información del lector, porque no aporta a la experiencia y elimina el riesgo de exponer datos personales.

Proceso y agilidad

Se eligió Kanban porque admite incorporar el cambio de la hora 6 como nuevas tareas sin replanificar ciclos. Se comparó con Scrum, que se descartó porque sus roles, ceremonias y sprints no tienen sentido para una persona en 12 horas. La especificación se hizo en los requisitos derivados de N01–N06; el desarrollo, por issues del tablero; la validación, con criterios de aceptación y pruebas automatizadas, y la evolución, incorporando la solicitud de cambio a partir

de `v1.0`.

Las prácticas ágiles fueron un tablero Kanban en GitHub Projects con límite de trabajo en curso y commits pequeños y frecuentes vinculados a los issues; se descartaron las reuniones diarias por no haber con quién sincronizar. Una tarea está terminada cuando cumple lo que pide la guía y fue revisada por el autor. El aprendizaje principal fue que trabajar dentro de OneDrive sobrescribía cambios con versiones anteriores; la acción de mejora fue mover el repositorio a una carpeta local y verificar cada archivo después de modificarlo. Las capturas del tablero están en el Anexo C.

Requisitos

Se especificaron 18 requisitos funcionales atómicos, cada uno con comportamiento observable y criterio de aceptación (Tabla 1). El detalle está en `docs/04_requisitos.md`.

Tabla 1

Requisitos funcionales por necesidad

Necesidad	Requisitos
N01 Historias en archivos y punto de inicio	RF01 Cargar historia; RF02 Iniciar partida
N02 Ver la escena y solo opciones compatibles	RF03 Mostrar escena; RF04 Mostrar solo opciones habilitadas; RF13 Informar bloqueo
N03 Consecuencias, destino y entradas inválidas	RF05 Aplicar efectos; RF06 Avanzar al destino; RF07 Rechazar entradas inválidas
N04 Recordar lo ocurrido	RF08 Recordar el estado de la partida
N05 Reconocer el final y reiniciar	RF09 Reconocer el final; RF10 Comenzar nueva partida
N06 Saber por qué un archivo no es válido	RF11 Validar antes de jugar; RF12 Informar la causa del error
Solicitud de cambio	RF14 Deshacer; RF15 Deshacer desde un final; RF16 Un solo nivel; RF17 Sin retroceso al iniciar o reiniciar; RF18 La entrada inválida conserva el retroceso

Tabla 2

Requisitos no funcionales

ID	Atributo	Medida	Verificación
RNF01	Claridad de errores	100 % de los 9 tipos de error de contenido producen un mensaje en español con la causa y el campo o ID, sin trazas de Python	Pruebas U4 (16 casos)
RNF02	Facilidad de uso	100 % de los textos de la interfaz en español; uso explicado en el README	Revisión de 12 textos en consola
RNF03	Confiabilidad	0 cierres inesperados ante texto, número fuera de rango, entrada vacía u opción no habilitada; el estado no cambia	Pruebas U3 y prueba manual en consola

El conflicto entre la escritora (retroceder siempre) y el diseñador (decisiones irreversibles) se documentó con una pregunta de aclaración que proponía un modo historia y un modo libre, y con el acuerdo provisional de no implementar retroceso en `v1.0`. La solicitud de cambio respondió que se deshace solo la última decisión (Universidad Tecnológica de Bolívar, 2026b). Se registraron cuatro supuestos: no se analizan todos los caminos al cargar; todos los usuarios leen español; el lector elige escribiendo el número de la opción, y el 0 deshace cuando es posible. Como defecto, varios requisitos iniciales no eran atómicos (por ejemplo, avanzar, aplicar efectos y rechazar entradas en uno solo); se corrigió dividiéndolos en RF05–RF07 y RF09–RF10.

Modelado

Los seis diagramas están en el Anexo A y su fuente editable en Mermaid en `docs/diagramas/`. El contexto muestra al motor como caja cerrada frente al lector, la escritora y los archivos JSON. Los casos de uso solo incluyen a los actores que usan el motor; llegar a un final y el bloqueo son extensiones de “Elegir una opción”. En las clases del dominio, `Partida` guarda el estado y la escena se divide en `EscenaIntermedia` y `EscenaFinal`. La secuencia muestra que la interfaz solo muestra y recibe, y que la partida valida en dos niveles. Los estados (en curso, en escena final y sin opciones disponibles) usan los mismos nombres que los métodos de `Partida`, para mantener un vocabulario único entre requisitos, modelos y código.

Arquitectura

Se eligió una arquitectura en capas: interfaz (`main.py`), reglas (`motor.py` y `modelo.py`), carga (`cargador.py`) y contenido (`data/`); cada capa solo usa a las de abajo.

Decisión 1, capas. Contexto: la escritora debe cambiar historias sin tocar el programa y el cambio de la hora 6 podía afectar cualquier parte. Alternativas: un monolito, rápido al inicio pero con todo mezclado, y MVC, cuyo controlador no aporta en una consola tan pequeña. Consecuencias: cargar otra historia no exige modificar `src/`, las reglas se prueban sin consola, y el costo es tener más archivos.

Decisión 2, validación en la partida. Contexto: el lector puede escribir cualquier cosa y el motor debe rechazar una opción no habilitada aunque se envíe su identificador. Alternativas: validar en la interfaz, que obligaría a repetirlo en otra interfaz, o en ambas, que duplica la regla. Elección: `elegir_por_numero()` valida formato y rango, y `elegir_opcion()` que la opción exista y esté habilitada. Consecuencia: la regla está en un solo lugar y se prueba sin consola.

La separación entre contenido, reglas e interfaz se comprueba con la segunda historia, *El último algoritmo*, que se juega con el mismo motor sin modificar ningún archivo de `src/`.

Implementación y pruebas

La decisión de diseño detallado fue separar la escena en dos clases que heredan de `Escena`. La alternativa descartada era un atributo `es_final`, que dejaría la regla escondida y obligaría a revisarla en todo el motor. Con dos clases, cada una exige al crearse la cantidad de opciones que le corresponde, la validación ocurre en un solo lugar y el formato JSON no cambia.

El motor usa solo la biblioteca estándar de Python (`json`, `sys`, `os`; licencia PSF) y las pruebas usan pytest 8.3.3 (licencia MIT), instalado dentro de un entorno virtual para no mezclarlo con otros proyectos (Python Software Foundation, 2024; pytest-dev, 2024). Las 58 pruebas automatizadas cubren U1–U6 e I1, incluidos los errores de contenido, el bloqueo y la segunda historia, y todas pasan (`docs/resultado_pruebas.txt`). La verificación manual de RNF02 y RNF03 está en `docs/09_trazabilidad.md`, con su captura en

`docs/images/verificacion_consola.png`.

Solicitud de cambio

El cliente pidió deshacer la última decisión revirtiendo todas sus consecuencias, con un solo nivel y sin rehacer; se especificó en RF14–RF18. El análisis de impacto mostró que solo cambiaron `motor.py` (atributo `respaldo` y métodos `puede_deshacer()` y `deshacer()`) y `main.py` (opción “0. Deshacer la última decisión”). El modelo, el cargador y las historias no se tocaron, lo que confirma la decisión de capas. Se actualizaron los requisitos, el alcance, los casos de uso, las clases, los estados y las pruebas (nueva U6; U3, U5 e I1 ajustadas).

Se guarda un solo respaldo con la escena y una copia de las variables, porque el cliente pide un nivel y una lista sería complejidad innecesaria; la copia evita que deshacer devuelva valores ya modificados. Se consideró guardar el ID de la escena en lugar del objeto, más coherente con el resto del motor; se mantuvo el objeto porque las escenas no cambian durante la partida y ambas opciones se comportan igual.

Trazabilidad

La matriz del Anexo B relaciona los 18 RF y los 3 RNF con su origen, los modelos, los archivos o métodos que los atienden y la prueba que los verifica por nombre. Solo RF03 y RNF02 se verifican manualmente, porque mostrar texto en pantalla no se comprueba con una prueba automática.

Aportes y fuentes

El autor dirigió el trabajo: tomó las decisiones de requisitos, modelado, arquitectura y del cambio, propuso la trama de la segunda historia y comparó con la guía todo lo producido antes de aprobarlo; la declaración completa, vinculada con archivos y commits, está en `docs/10_aportes.md`. La redacción, el código y las pruebas se elaboraron con apoyo de Claude (Anthropic, 2026) y la redacción de la segunda historia se pulió con ChatGPT (OpenAI, 2026); cada uso, su propósito y su verificación están en `docs/fuentes.md`.

Referencias

Anthropic. (2026). *Claude* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Chiquillo Torres, A. (2026). *sistema_de_historias_interectativas* [Repositorio de código]. GitHub. https://github.com/8alanct-hub/sistema_de_historias_interectativas

OpenAI. (2026). *ChatGPT* [Modelo de lenguaje de gran tamaño]. https://chatgpt.com

pytest-dev. (2024). *pytest documentation* (Versión 8.3). https://docs.pytest.org

Python Software Foundation. (2024). *Python 3.11 documentation*. https://docs.python.org/3.11/

Universidad Tecnológica de Bolívar. (2026a). *Parcial práctico: La última transmisión* [Guía de la actividad].

Universidad Tecnológica de Bolívar. (2026b). *Solicitud de cambio del parcial* [Documento de la actividad].